

Тема 1.2 Оператор ветвления

Условные операторы позволяют в зависимости от значения логического выражения выполнить отдельный участок программы или, наоборот, не выполнить его. Логические выражения возвращают только два значения: **True** (истина) или **False** (ложь), которые ведут себя как целые числа 1 и 0 соответственно:

```
print(True + 2) # Эквивалентно 1 + 2
3
print(False + 2) # Эквивалентно 0 + 2
2
```

Логическое значение можно сохранить в переменной:

```
x = True
y = False
print(x, y)
(True, False)
```

Любой объект в логическом контексте может интерпретироваться как истина (**True**) или как ложь (**False**). Для определения логического значения можно использовать функцию **bool()**.

Значение **True** возвращает следующие объекты:

- любое число, не равное нулю:

```
print(bool(1), bool(20), bool(-20))
(True, True, True)
print(bool(1.0), bool(0.1), bool(-20.0))
(True, True, True)
```

- не пустой объект:

```
print(bool("0"), bool([0, None]), bool((None, )), bool({'x': 5}))
(True, True, True, True)
```

Следующие объекты интерпретируются как **False**:

- число, равное нулю:

```
print(bool(0), bool(0.0))
(False, False)
```

- пустой объект:

```
print(bool(""), bool([]), bool(()))
(False, False, False)
```

- значение **None**:

```
print(bool(None))
False
```

4.2 Операторы сравнения в языке Python

Операторы сравнения используются в логических выражениях. Перечислим их:

- **==** (равно):

```
1 == 1, 1 == 5
```

(True, False)

- != (не равно):

1 != 5, 1 != 1

(True, False)

- < (меньше):

1 < 5, 1 < 0

(True, False)

- > (больше):

1 > 0, 1 > 5

(True, False)

- <= (меньше или равно):

1 <= 5, 1 <= 0, 1 <= 1

(True, False, True)

- >= (больше или равно):

1 >= 0, 1 >= 5, 1 >= 1

(True, False, True)

- in (проверка на вхождение в последовательность):

'Строка' in 'Строка для поиска' # Строки

True

2 in [1, 2, 3], 4 in [1, 2, 3] # Списки

(True, False)

2 in (1, 2, 3), 4 in (1, 2, 3) # Кортежи

(True, False)

Оператор in можно также использовать для проверки существования ключа словаря:

'x' in {'x': 1, 'y': 2}, 'z' in {'x': 1, 'y': 2}

(True, False)

- not in (проверка на невхождение в последовательность):

'Строка' not in 'Строка для поиска' # Строки

False

not in [1, 2, 3], 4 not in [1, 2, 3] # Списки

(False, True)

2 not in (1, 2, 3), 4 not in (1, 2, 3) # Кортежи

(False, True)

- is - проверяет, ссылаются ли две переменные на один и тот же объект. Если переменные ссылаются на один и тот же объект, то оператор is возвращает значение True:

x = y = [1, 2]

x is y

True

x = [1, 2]

y = [1, 2]

x is y

False

- is not - проверяет, ссылаются ли две переменные на разные объекты. Если это так, возвращается значение True:

```
x = y = [1, 2]
```

```
x is not y
```

False

```
x = [1, 2]
```

```
y = [1, 2]
```

```
x is not y
```

True

Значение логического выражения можно инвертировать с помощью оператора not:

```
x = 1
```

```
y = 1
```

```
x == y
```

True

```
not (x == y), not x == y
```

(False, False)

Если переменные **x** и **y** равны, то возвращается значение **True**, но так как перед выражением стоит оператор **not**, выражение вернет **False**. Круглые скобки можно не указывать, поскольку оператор **not** имеет более низкий приоритет выполнения, чем операторы сравнения.

В логическом выражении можно указывать сразу несколько условий:

```
x = 10
```

```
1 < x < 20, 11 < x < 20
```

(True, False)

Несколько логических выражений можно объединить в одно большое с помощью следующих операторов:

- and (логическое И). Если **x** в выражении **x and y** интерпретируется как False, то возвращается **x**, в противном случае - **y**:

```
1 < 5 and 2 < 5 # True and True == True
```

True

```
1 < 5 and 2 > 5 # True and False == False
```

False

```
1 > 5 and 2 < 5 # False and True == False
```

False

```
10 and 20, 0 and 20, 10 and 0
```

(20, 0, 0)

- or (логическое ИЛИ). Если **x** в выражении **x or y** интерпретируется как False, то возвращается **y**, в противном случае - **x**:

```
1 < 5 or 2 < 5 # True or True == True
```

True

```
1 < 5 or 2 > 5 # True or False == True
```

True

```
1 > 5 or 2 < 5 # False or True == True
```

```
True
```

```
10 or 20, 0 or 20, 10 or 0
```

```
(10, 20, 10)
```

```
0 or "" or None or [] or "s"
```

```
's'
```

Следующее выражение вернет **True** только в случае, если оба выражения вернут **True**:

```
x1 = x2 and x2 != x3
```

А это выражение вернет **True**, если хотя бы одно из выражений вернет **True**:

```
x1 = x2 or x3 = x4
```

Перечислим операторы сравнения в порядке убывания приоритета:

- <, >, <=, >=, ==, !=, is, is not, in, not in.
- not (логическое отрицание).
- and (логическое И).
- or (логическое ИЛИ).

4.3 Оператор ветвления if...else

Оператор ветвления **if...else** позволяет в зависимости от значения логического выражения выполнить отдельный участок программы или, наоборот, не выполнить его. Оператор имеет следующий формат:

```
if <Логическое выражение>:  
    <Блок, выполняемый, если условие истинно>  
elif <Логическое выражение>:  
    <Блок, выполняемый, если условие истинно>  
]  
else:  
    <Блок, выполняемый, если все условия ложны>  
]
```

Блоки внутри составной инструкции выделяются одинаковым количеством пробелов (обычно четырьмя пробелами). Концом блока является инструкция, перед которой расположено меньшее количество пробелов. В некоторых языках программирования логическое выражение заключается в круглые скобки. В языке **Python** это делать необязательно, но можно, т. к. любое выражение может быть расположено внутри круглых скобок. Тем не менее, круглые скобки следует использовать только при необходимости разместить условие на нескольких строках.

Для примера напомним программу, которая проверяет, является введенное пользователем число четным или нет.

```
# -*- coding: utf-8 -*-  
x = int(input("Введите число: "))  
if x % 2 == 0:
```

```
print(x, " - четное число")
else:
    print(x, " - нечетное число")
```

После проверки выводится соответствующее сообщение (рисунок 1):

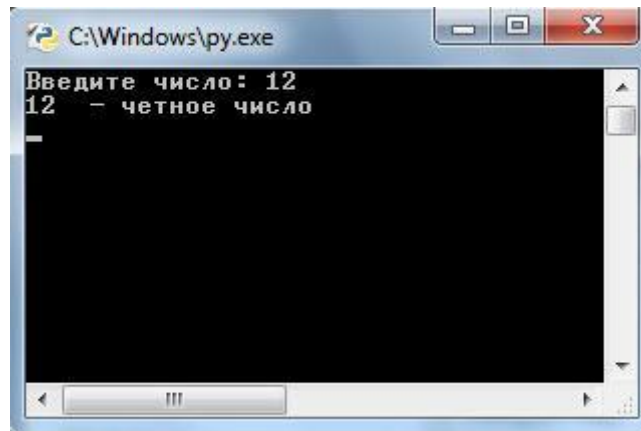


Рисунок 1 – Результат работы скрипта

Если блок состоит из одной инструкции, то эту инструкцию можно разместить на одной строке с заголовком:

```
# -*- coding: utf-8 -*-
x = int(input("Введите число: "))
if x % 2 == 0: print(x, " - четное число")
else: print(x, " - нечетное число")
```

В этом случае концом блока является конец строки. Это означает, что можно разместить сразу несколько инструкций на одной строке, разделяя их точкой с запятой:

```
# -*- coding: utf-8 -*-
x = int(input("Введите число: "))
if x % 2 == 0: print(x, end=" "); print(" - четное число")
else: print(x, end=" "); print(" - нечетное число")
```

Следующий код читается намного проще, чем предыдущий:

```
# -*- coding: utf-8 -*-
x = int(input("Введите число: "))
if x % 2 == 0:
    print(x, end=" ")
    print(" - четное число")
else:
    print(x, end=" ")
    print(" - нечетное число")
```

Оператор **if...else** позволяет проверить сразу несколько условий. Рассмотрим это на следующем примере:

```
# -*- coding: utf-8 -*-
print("""Какой операционной системой вы пользуетесь?
1 - Windows 10
2 - Windows 11
```

```

3 - Windows Vista
4 - Windows XP
5 - Другая""")
os = input("Введите число, соответствующее ответу: ")
if os == "1":
    print("Вы выбрали: Windows 10")
elif os == "2":
    print("Вы выбрали: Windows 11")
elif os == "3":
    print("Вы выбрали: Windows Vista")
elif os == "4":
    print("Вы выбрали: Windows XP")
elif os == "5":
    print("Вы выбрали: другая")
elif not os:
    print("Вы не ввели число")
else:
    print("Мы не смогли определить вашу операционную систему")

```

С помощью инструкции **elif** можно определить выбранное значение и вывести соответствующее сообщение. Обратите внимание на то, что логическое выражение не содержит операторов сравнения:

```

elif not os:
    .
Такая запись эквивалентна следующей:
elif os == "":

```

Проверка на равенство выражения значению **True** выполняется по умолчанию. Поскольку пустая строка интерпретируется как **False**, то инвертируем возвращаемое значение с помощью оператора **not**.

Один условный оператор можно вложить в другой. В этом случае отступ вложенной инструкции должен быть в два раза больше:

```

# -*- coding: utf-8 -*-
print("""Какой операционной системой вы пользуетесь?
1 - Windows 8
2 - Windows 7
3 - Windows Vista
4 - Windows XP
5 - Другая""")
os = input("Введите число, соответствующее ответу: ")
if os != "":
    if os == "1":
        print("Вы выбрали: Windows 10")
    elif os == "2":
        print("Вы выбрали: Windows 11")
    elif os == "3":
        print("Вы выбрали: Windows Vista")
    elif os == "4":

```

```

    print("Вы выбрали: Windows XP")
elif os == "5":
    print("Вы выбрали: другая")
else:
    print("Мы не смогли определить вашу операционную систему")
else:
    print("Вы не ввели число")

```

Оператор **if...else** имеет еще один формат:

<Переменная> = <Если истина> **if** <Условие> **else** <Если ложь>

Пример:

```
print("Yes" if 10 % 2 == 0 else "No")
```

Результат:

Yes

```
s = "Yes" if 10 % 2 == 0 else "No"
```

```
print(s)
```

Результат:

'Yes'

```
s = "Yes" if 11 % 2 == 0 else "No"
```

```
print(s)
```

Результат:

'No'

4.4 Пример решения задачи в языке Python

Задача. Напишите программу, которая запрашивает у пользователя количество еды. Затем программа проверяет, достаточно ли еды для насыщения. Если введенное количество еды больше нуля, программа выводит сообщение о насыщенности в зависимости от количества.

```

# Вводим количество еды
food = int(input("Введите количество еды:"))
# Проверяем, достаточно ли еды для насыщения
if food > 0:
    # Если еда есть, проверяем количество
    if food >= 3:
        print("У вас достаточно еды. Вы не голодны.")
    elif 1 <= food < 3:
        print("У вас мало еды. Вы чувствуете голод.")
    else:
        print("У вас совсем нет еды. Вы очень голодны.")
else:
    # Если еды нет, выводим сообщение о голоде
    print("У вас нет еды. Вы очень голодны.")

```

Программа ветвится в зависимости от количества еды:

- если еды достаточно (3 и более единиц), выводится сообщение о том, что персона не голодна;
- если еды недостаточно (от 1 до 2 единиц), выводится сообщение о том, что персона чувствует голод;
- если еды вообще нет, выводится сообщение о том, что персона очень голодна.